What survives when a paper leaves the lab: re-testing 22 robot-learning papers on a €1,350 rig, and auditing our own pre-registration

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@gmail.com*

Abstract

Most robot-learning results are shown in simulation, on costly arms or on data-centre GPUs. We wanted to know which of them hold on hardware a student can buy, so we re-tested the central claim of 22 papers on a low-cost SO-101 arm and a Jetson Orin NX. Each re-test ran the cheapest baseline first and had its pass mark fixed before the run. Of the 37 claims taken from the papers, 12 reproduce and 18 fail, and the rest are not proven or not tested. Three proposed classes cover the failures we saw: methods that miss a real-time budget on the edge GPU, zero-shot judges that a trivial signal beats, and latent planners that the commanded joint positions beat. We audited the study itself as well. Every published number was rebuilt from its stored rows, the early gates were recomputed with one statistic, and commit order shows which bars came before their results. Most did, but for 29 of 89 gates the order cannot be shown. The class assignment waits for a blind second reader. All compute cost $45.10.

1 The register

A claim shown on a Franka-class arm, in simulation or on a data-centre GPU does not tell you whether it holds on the hardware most students and small labs own. Ours is that kind of hardware: an SO-101 arm and a Jetson Orin NX 16 GB, €1,350 for the parts with a stated price (2 small parts are not priced yet). Few of the papers we re-tested were shown on such hardware. 9 of the 22 were shown in simulation and 7 on rigs above $20k, against 3 at $5–20k and 2 below $5k; 1 does not state its setup.

The register has one entry per paper. It records the claim as the paper states it, the pass mark we fixed for our rig, our result and a verdict, and every number in it can be recomputed from published rows. Two rules govern each entry. Before a learned model gets credit, the trivial signal it must beat (copy-last, the commanded joints, episode length) runs on the same rows. And the pass mark is committed before the result, a rule whose record §2 audits.

We picked the papers for relevance to the stack this rig runs: latent world models, VLA policies, failure monitors and judges, action chunking and edge latency. Each went into the register before its test. They are not a sample of the field (§5), and the register file already lists the papers scheduled next.

2 Method, and an audit of our own pre-registration

Each gate of the experiment log has a bar and a result. For each, we find in git history the first commit that contains the bar and the first that contains the result, and classify the gate by commit ancestry alone (`git merge-base --is-ancestor`). Anyone with a clone can check ancestry, which is not true of a note or a timestamp, so we use neither. Of the 89 gate rows in the log on 2026-10-31, **58 are bar-first** and 0 result-first. The others are 6 same-commit, 23 with no bar and 2 with a bar but no result yet. The log has since grown to 93 rows (61 bar-first, 7 same-commit, 0 result-first, 23 no-bar, 2 not-run). Same-commit means the bar and the result first appear in one commit of this repository, which says nothing about which came first.

Our first pass searched the text of the log and got a different picture: 36 bar-first, 31 same-commit, 19 no-bar and 3 unresolved. The log often received a bar and its result in one commit even when the bar had been written earlier, in a protocol, plan, scorer or specification. Tracing each same-commit gate to that file moved 21 to bar-first and 4 to no-bar, because their bar cell declares none, and left 6. Each upgrade rests on a bar text found in an earlier commit of a file tied to the gate, and the audit table gives the evidence line so a reader can check it. We did not re-audit the rows that were already bar-first for a result hidden elsewhere in the history. One gate added on 2026-11-01, the reward-model progress test, reads same-

*Every number with its source file: `github.com/ravediamond/paper-register-reproducibility`.

commit by text search although its protocol (`6241570`) predates the log commit. We have not audited its result commit, so it stays same-commit as an open candidate.

The early gates mixed point estimates with assorted intervals. We recomputed 10 of them from stored rows with one statistic, the episode bootstrap (2,000 resamples, percentile 95 % CI), and found 3 verdicts whose bar falls inside the interval. The judge accuracy is the clearest case: 94.2 % on 87 episodes against a bar of 95 %, a fail on the point estimate that an interval of 89.6–98.9 % cannot settle. The oscillation rate among 25 failed episodes is a pass on the point (24.0 % against a bar of 10 %) with a lower bound of 8.0 %. A judge threshold's false-intervention rate, 2.4 % on 42 episodes against a ceiling of 5 %, is a pass whose interval reaches 7.1 %, and since the threshold was fitted to meet that rate, the interval is only descriptive. The register keeps the point verdicts and marks these three as fragile.

Every published number was recomputed from its published rows, and all 29 of 29 write-ups pass. Rerunning cold is harder. From a fresh clone with no credentials, 6 gates rerun and reproduce, 5 of them exactly. Another 12 are blocked by one software environment we never published, and the remaining 11 need a larger GPU, a private checkpoint, the arm or a step the write-up leaves out. We also checked the register's citations and paper-side fields against the papers' PDFs and merged the fixes. Of the 38 arXiv ids the published register names, those missing from our papers mirror belong to scheduled tests. One tested paper has no arXiv id and was not checked against its PDF.

3 Results

Table 1 lists the register. Counted by paper, 4 of the 22 reproduce and 13 fail, while 3 are not proven, 1 was not tested and 1 was withdrawn. The register holds 45 claim rows, 42 of them tested. 8 of these rows test our own baselines rather than a paper's claim, for example "the commanded joints beat the world model", and 5 of those reproduce. On the 37 claims taken from papers, 12 reproduce and 18 fail. Over all rows the count is 17 reproduce, 20 fail, 3 not proven, 4 not tested and 1 withdrawn.

4 Three failure classes

We propose three classes for the 20 failed claim rows, named after what beat the method. In class A it is the clock: the method may work, but on the edge GPU it misses the real-time budget the claim needs. An adaptive chunking method, for example, takes 4,751 ms per decision against a budget of 298 ms. Class B is zero-shot judges, models used without training on this rig to judge success, failure or progress, that lose to a trivial signal. A learned reward model detects failures at AUROC 0.818, where episode length alone reaches 0.938. Class C is learned latents used to plan, rank or track progress that do worse than the commanded joints or a copy-last or action-only baseline. A latent world model ranks the executed chunk first in 0.60 of anchors, below its bar of 0.8 and below the 0.68–0.90 of an action-only baseline.

These three examples illustrate the classes and count nothing. Under the protocol committed before any reading, per-row labels count only once a blind second reader, labelling the same 20 rows on the same form, agrees with the first pass at Cohen's $\kappa \geq 0.6$. The protocol and the form are in the repository, and the second reading has not happened yet.

5 Limitations

Everything here comes from one rig, one arm and one operator, so a failure is a finding about this rig and not a refutation of the paper in its own setting. One person labelled the verdicts, at rest and not always blind to the arm; a second labeller would show whether any verdict flips, and the class check of §4, the only independent reading planned, has not run. For 29 of the 89 gates (6 same-commit, 23 with no bar) this repository cannot show that the bar came first, and the 21 upgrades of §2 depend on matching a bar text. Under the bootstrap, 3 verdicts are fragile, and only 6 gates have been rerun cold. The papers were chosen for relevance to one stack, so the reproduction rate describes this register and not the field.

6 Related work

Work on the same arm evaluates policies. A standardized real-world benchmark reports a taxonomy of failures and recoveries for VLA policies [13], and a fleet of SO-101 cells evaluates policies at scale [14]. The vla-eval tool decouples VLA inference from 14 simulation benchmarks through a WebSocket protocol and Docker isolation [8], and a benchmark of demonstration quality studies one method family empirically [22]. We test something else: the claims behind methods (judges, world models, latency) against bars fixed before the run, with our own pre-registration audited. The closest statement of our thesis [27] scores VLA and world-action models on five dimensions, among them reproduction evidence and deployment affordability. Its reproduction-evidence score counts five audit milestones (configuration mapping, task records, task coverage, training records, seeds), and its authors state that it is not a probability that a result is correct. We give one pass or fail per claim on a €1,350 rig instead.

7 Conclusion

On a €1,350 rig, 12 of 37 tested paper claims reproduce under bars fixed for this hardware before the run. Compute was cheap, $45.10 for all of it. The hard part

paper	their rig	claim tested	bar	our result	verdict
LeWorldModel [9]	sim	A small JEPA world model learns the arm's dynamics well enough to plan with it.	Dynamics: beat copy-last, every corruption ≥ 1.3 at $K = 30$. Planning: rank the executed chunk first among 8 in ≥ 0.8 of anchors.	dynamics 0.31 vs 0.89 copy-last (reproduces); planning top-1 0.60 vs 0.68–0.90 for an action-only baseline (fail)	fail
DINO-WM [1]	sim	A predictor on frozen DINOv2 patches plans better than one on a learned latent.	G1b top-1 ≥ 0.8 at some target distance.	arm-pose R^2 0.92 (vs 0.57 for W1); G1b top-1 0.55–0.61; no-model baseline 0.68–0.90	fail
RoboJEPA [20]	>$20k	Pretrained RoboJEPA predicts our latents, uses the actions, and runs on the robot's computer.	9-step L1 ≥ 15 % below copy-last (300M) and ≥ 5 % below shuffled actions; 2-view encoder ≤ 200 ms on the Orin.	11.8 % below copy-last (300M; 1B 15.8 % on 30 clips); 9–10 % below shuffled; encoder 526 ms on the Orin NX	fail
H-JEPA [19]	sim	A slow second level makes latent distance track task progress.	Progress monotonicity Spearman $\rho \leq -0.80$; false stalls ≤ 10 %.	level-2 ρ -0.73; false stalls 52 %	fail
D-JEPA [18]	<$5k	A decision-aligned ranking operator on a frozen world model picks better action chunks.	G1b regret ≤ 0.10 and $\leq$ the action-only baseline + 0.05.	regret 0.263 (W1 alone 0.263; action-only -0.007)	not tested
ContactGuard [16]	$5–20k	An event-anchored probe on imagined latents predicts grasp failure better than the current latent.	AUROC ≥ 0.85 and ≥ 0.05 above the current latent.	0.70 imagined vs 0.82 current	fail
FAIL-Detect and SAFE [2] [5]	>$20k	A conformal band on a learned failure score keeps false alarms at α while catching failures.	Successes flagged $\leq 1/20$; failures flagged ≥ 50 %.	22.7 % of successes flagged at $\alpha = 5$ %; 46 % of failures caught	fail
ReAP [21]	$5–20k	A risk head on an action-conditioned latent warns before failures.	$\geq 5/8$ failures caught at $\leq 1/20$ false interventions; action-conditioned iff true $\geq$ shuffled + 0.05.	v1 AUROC 0.93 came from time and length; v2 0.789 vs 0.708 shuffled, 3/8 at 5/42	fail
VLA-Corrector [17]	sim	A verifier on observable history flags a failing rollout early enough to correct it.	$\geq 5/8$ failures at $\leq 1/20$ false interventions.	the paper's verifier family on our rollouts (a GRU on pooled RGB statistics and actions): 2/8 caught at 3/42 false interventions, AUROC 0.59; cosine drift of the W1 latent 0/8, AUROC 0.30	fail
Robometer [7]	>$20k	A 4B reward model judges SO-101 rollouts zero-shot.	Pooled AUROC ≥ 0.85 and within-policy ≥ 0.8 on ≥ 5 of 7 policies.	pooled AUROC 0.818 on 2099 ArmnetBench rollouts; episode length alone 0.938	fail
SmolVLA [3]	<$5k	A 450M VLA succeeds on an SO-101 task and can run on low-cost hardware.	10-trial operator-judged gate (no numeric pass mark registered); chunk inference within the async budget (667 ms).	9/10 on bowls; 974 ms in PyTorch, 238 ms with vla.cpp on the Orin NX	reproduces

Table 1: The register: one row per paper, generated from the register file (claim, bar and result as recorded, including our internal model and gate names); each re-tested paper is cited beside its name.

paper	their rig	claim tested	bar	our result	verdict
TurboVLA [15]	sim	Our claim that the architecture made the policy ignore its instruction.	Task swap $\geq 5°$ and per-layout $\geq$ 75 % $\rightarrow$ our port was the bug; $< 1°$ with the control $\geq 0.095 \rightarrow$ the data.	upstream's own trainer on our data: task swap 0.020°, per-layout 38.9 %; control, their released LIBERO checkpoint run here: 0.105 ($\approx 5.5°$ of pan)	withdrawn
VLA-JEPA [6]	sim	VLA-JEPA trains cheaply on SO-101 data and runs on the robot.	10-trial operator-judged gate (no numeric pass mark registered); for reranking (H0): oracle / mean error $\leq$ 0.7 and N = 8 $\leq 1.5\times$ the latency.	9/10 for $9.7; 355 ms per chunk on the Orin NX; reranking no-go (0.76, 1.84×)	reproduces
REKI [23]	sim	Policies without memory oscillate between poses at a measurable rate.	$\geq$ 10 % of failed episodes oscillate (their criterion).	20.3 % of all episodes, 24.0 % of failures; REKI trained on my data: 19/20 + 19/20 on the arm, oscillates in 1/40	reproduces
Real-Time Chunking [4]	unstated	Conditioning a new chunk on the executing one removes the jump at chunk boundaries.	Best gradient-free method cuts the median jump $\geq$ 50 % against naive async, drift $\leq$ +10 %, within 667 ms.	prefix overwrite cuts the jump 71–73 % against naive async at +0.4 ms; tuned LeRobot identity mode is close	not proven
HWM (hierarchical latent world models) [10]	>$20k	A high-level world model over learned macro-actions beats the low-level model beyond about 1 s (their Fig. 6 crossover).	Crossover in [1.0, 2.0] s; at 2 s high-level error $\leq$ 0.95 × low-level; learned macro-actions $\geq$ net joint displacement on cosine.	no crossover: W1 0.382 ± 0.005 $\rightarrow$ 0.563 ± 0.007, high level 0.769 ± 0.009 $\rightarrow$ 0.759 ± 0.009 (0.73–2 s); cosine 0.608 ± 0.007 vs 0.551 ± 0.006	not proven
LeJEPA identifiability [12]	sim	A SIGReg JEPA trained on goal-directed data recovers the true latent state only partially and unevenly.	W1's mean R^2 from latent to joints < 0.80; anisotropy (max − min per-joint R^2) ≥ 0.20; the higher-R^2 model has the higher G1b top-1.	W1 mean R^2 0.565, anisotropy 0.639 (shoulder lift 0.91, wrist flex 0.27); frozen DINOv2-S 0.867 / 0.301	reproduces
SURVEYOR	sim	A threshold on the imagined-vs-reached latent distance accepts good plans and rejects bad ones ($\tau = 0.20$).	τ below the 10th percentile of non-equivalent pairs and ≤ 0.40; as a monitor $\geq$ 5/8 failures at $\leq$ 2/42; $\geq$ 0.05 AUROC over a shuffled drafter.	$\tau = 1.249$ on W1 latents of 69 real rollouts; as a monitor 3/8 at 6/42 (S = 30), AUROC 0.756 vs shuffled 0.560	fail
Adaptive Action Chunking (AAC) [11]	>$20k	Chunk length chosen from the entropy of N sampled chunks follows the task's phases (long in transport, short at the grasp), lowers the boundary jump, and costs little with batched sampling.	median h* in transport $\geq$ grasp window + 2; jump with AAC + overwrite $\leq$ fixed + overwrite at d = 8; an N = 20 decision $\leq$ 298 ms on the Orin.	h* 2 in transport and 2 at the grasp (86 % of decisions at the floor); jump 0.973 vs 0.940; N = 20 decision 4,751 ms (sequential) on the Orin	fail
PACT-VLA [24]	$5–20k	Picking among N sampled chunks by their flow residual under the pre-commit observation fits what happens next better than a random or stale pick, with no added control-step delay.	PACT beats a random pick and the stale-observation residual at d = 8 (CIs < 0); its delay on the Orin $\leq$ the naive delay; at that delay it still beats a single sample.	offline $-0.27°$ vs random and vs the stale residual (both CIs < 0); on the Orin 1,200 ms per cycle, 37 control ticks vs 8	fail
Horizon × depth (Euler steps vs execution horizon) [25]	>$20k	With long open-loop commitment, cutting a flow-matching VLA's Euler steps from 10 to 2 contracts the action cloud around an unchanged centroid, saves most of the inference time and avoids edge-GPU thermal throttling.	median R(2)/R(10) $\leq$ 0.70 with the centroid within 0.05 relative and 3°; N = 2 latency $\leq 0.85 \times$ N = 10 on the Orin, N = 10 within 5 % of 238 ms.	R(2)/R(10) 0.78 (CI 0.73–0.81), centroid 0.018 and 2.35°; 196.1 vs 237.5 ms per chunk (0.83, vision 151 ms); no throttling at N = 10 on the Orin, at MAXN_SUPER or 25 W	not proven
REMORA [26]	>$20k	Reading a progress distribution out at the supported peak nearest the previous estimate steadies the estimate and lets it fall during real setbacks, with no worse detection of the stages.	Against per-frame argmax: premature not higher and missed $\leq$ +2 points (P1); flips halved, CI upper $\leq$ 0.7 (P2); a decrease inside $\geq$ 50 % of the reversal windows with $\leq$ 20 % false regressions (P3).	premature 0.22 vs 0.25, missed equal (P1 pass); flips 0.70 × argmax, CI 0.48–0.94 (P2 fail); setback recall 2/7, false 0.52 (P3 fail); MAE 0.184 vs median 0.218	fail

Table 1, continued.

was order, keeping every bar ahead of its result, and our own history has 29 of 89 gates where we cannot prove we did. The next papers are scheduled in the register file, and every number here can be rebuilt from the repository (`github.com/ravediamond/paper-register-reproducibility`).

References

[1] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, 2024. arXiv:2411.04983.

[2] Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies, 2025. arXiv:2503.08558.

[3] SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[4] Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[5] SAFE: Multitask Failure Detection for Vision-Language-Action Models, 2025. arXiv:2506.09937.

[6] VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model, 2026. arXiv:2602.10098.

[7] Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons, 2026. arXiv:2603.02115.

[8] vla-eval: A Unified Evaluation Harness for Vision-Language-Action Models, 2026. arXiv:2603.13966.

[9] LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, 2026. arXiv:2603.19312.

[10] Hierarchical Planning with Latent World Models, 2026. arXiv:2604.03208.

[11] Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161.

[12] When Does LeJEPA Learn a World Model?, 2026. arXiv:2605.26379.

[13] Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[14] ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

[15] TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM, 2026. arXiv:2607.27205.

[16] ContactGuard: Pre-Contact Execution Monitoring with Action-Conditioned Latent World Models, 2026. arXiv:2608.13438.

[17] VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. arXiv:2609.06508.

[18] D-JEPA: A Decision-Aligned Latent World Model, 2026. arXiv:2609.24749.

[19] H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning, 2026. arXiv:2610.06805.

[20] RoboJEPA: Scaling Robotic Latent World Models, 2026. arXiv:2610.10515.

[21] ReAP: From Visual Failure Recognition to Action-Conditioned Risk Prediction for Runtime Failure Monitoring, 2026. arXiv:2610.16309.

[22] What Matters in Quality-Aware VLA Pretraining: A Benchmark and Empirical Study, 2026. arXiv:2610.18869.

[23] REKI: Registering Knowledge in VLAs, 2026. arXiv:2610.22178.

[24] PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

[25] Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[26] Tracking Task Progress Through Errors and Recovery:A Benchmark and Recurrent Distributional Model, 2026. arXiv:2610.39125.

[27] Xin Ma et al. RoboSeed: Multidimensional Evaluation of Vision-Language-Action and World-Action Models, 2026. arXiv:2610.40026.